03603332 OPERATING SYSTEMS · GROUP PROJECT · DEADLOCKS

Deadlock Detector

Find deadlocks without changing the code.

```
$ LD_PRELOAD=./build/libdetect.so ./your_program
```

Supawit Chaiyanusak
6730300591

Chayanin Iamsakul
6730300094

Pattanadech Mahimuang
6730300850

Thatthorn Jiamton
6730300779

Phutawan Supkerd
6730300884

THE PROBLEM

Two threads. Two locks. Stuck forever.

Thread 1

moves money Alice → Bob

t0	lock(Alice)	✓ got it
t2	lock(Bob)	⏳ waits

Thread 2

moves money Bob → Alice

t1	lock(Bob)	✓ got it
t3	lock(Alice)	⏳ waits

No crash. No error. 0% CPU. It just stops.

DEADLOCK BASICS

4 Coffman conditions

01

Mutual exclusion

Only one thread can hold a lock.

02

Hold and wait

Each thread holds one account. It wants the other.

03

No preemption

Nobody can take a lock away.

04

Circular wait

Thread 1 waits for Thread 2. Thread 2 waits for Thread 1.

All 4 true → deadlock. Break one → no deadlock.

OUR GOAL

Make the silent program speak.

✓ Which threads

✓ Which locks

✓ Which code lines

No code change

No rebuild

```
$ LD_PRELOAD=./build/libdetect.so ./your_program
```

BIG PICTURE

Four steps

STEP 1

Catch

every lock call

`interpose.c`

→

STEP 2

Record

who holds, who waits

`state.c`

→

STEP 3

Check

for a loop every 500 ms

`detector.c`
`graph.c`

→

STEP 4

Report

threads, locks, code lines

`report.c`

STEP 1 — CATCH

LD_PRELOAD puts us in front

Your program
`pthread_mutex_lock(&A)`

↓

libdetect.so — our library

```
1  note "waiting for A"
2  call the real lock
3  note "got A"
```

↓

libc.so.6 — the real lock

↓

Linux kernel — we do not touch it

LD_PRELOAD loads our library first.

So our lock function runs before the real one.

- `pthread_mutex_lock`
- `pthread_mutex_trylock`
- `pthread_mutex_unlock`

STEP 2 — RECORD

Two tables

holds[]

Who holds each lock?

LOCK	HELD BY
A	Thread 1
B	Thread 2

waits[]

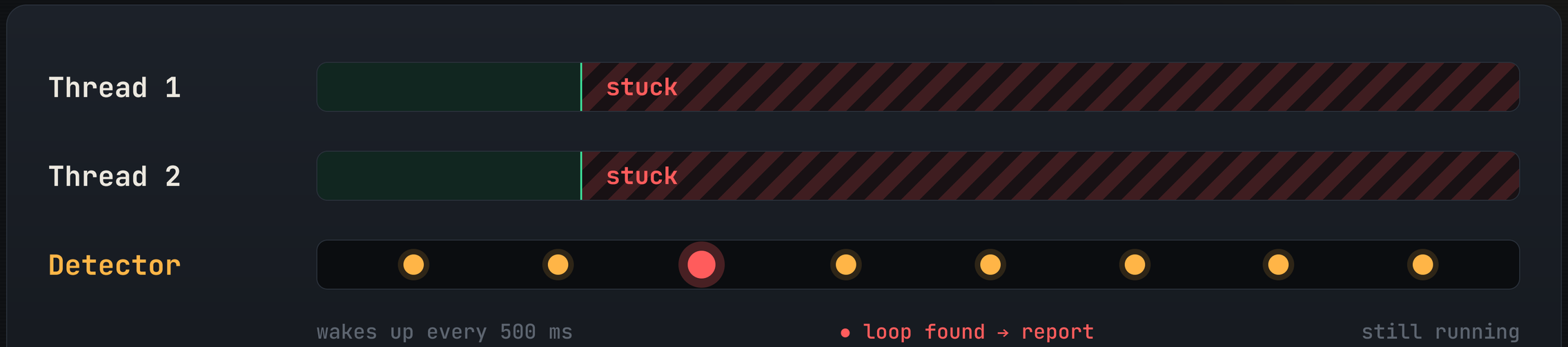
Who waits for which lock?

THREAD	WAITS FOR
Thread 1	B
Thread 2	A

Fixed size. **No malloc.** ← malloc uses locks inside → it would call our code again → `in_dd` flag stops the loop

STEP 3 — CHECK

The detector thread



OWN THREAD

It copies the tables and looks for a loop.

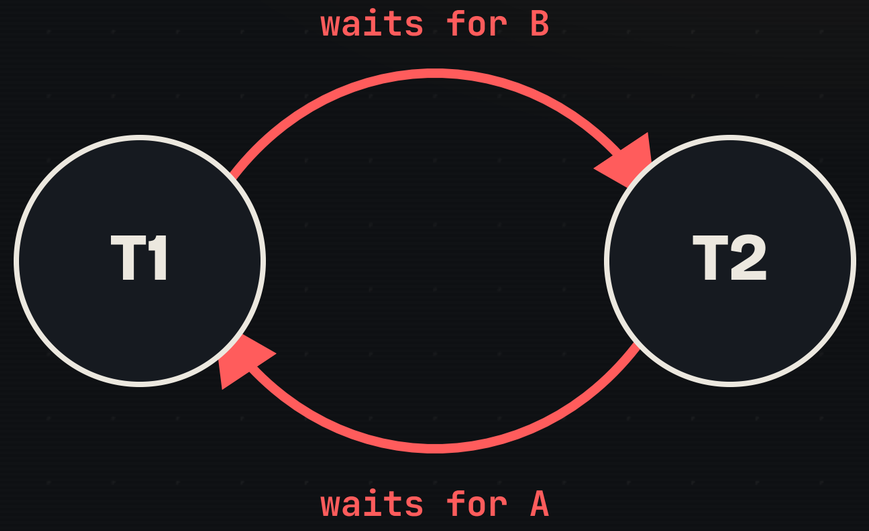
NEVER STUCK

It never uses the program's locks.

STEP 3 — CHECK

Wait-for graph: find the loop

TIME	WHAT HAPPENS	GRAPH
t0	T1 locks A	A held by T1
t1	T2 locks B	B held by T2
t2	T1 wants B → waits	T1 → T2
t3	T2 wants A → waits	T2 → T1
≤ 500 ms	detector finds the loop → report	



loop = circular wait = deadlock

One lock at a time → **max one arrow out** → just follow the arrows. finds every loop · up to 16

STEP 4 — REPORT

The real output

```
[DEADLOCK DETECTED] 2 threads in cycle

  Thread 514
      holds   mutex 0x56fc6b20b070   at bank.c:89
      waiting mutex 0x56fc6b20b030   at bank.c:94
  Thread 513
      holds   mutex 0x56fc6b20b030   at bank.c:65
      waiting mutex 0x56fc6b20b070   at bank.c:70

  Cycle: T514 → T513 → T514
  Hint : acquire locks in a consistent global order
```

holds got this lock at this line

waiting stuck at this line

Cycle the loop

Hint how to fix it

code lines: dladdr + addr2line · needs -g and PIE

DD_DOT_OUT → picture (Graphviz)

HOW WE TESTED

Tests that must pass

Must deadlock + report

bank
2 threads, money transfer

sample_webapp
looks like real code

cycle3
3 threads in one loop

two_pairs
2 loops at once

Must stay quiet

no_deadlock
same lock order → no report

condvar
condition variable → no report

Hard cases

robust
lock owner died

errcheck
wrong thread unlocks

longcycle
70 threads in one loop

lockorder
opposite order, no deadlock → warning

PTHREAD_BARRIER, NOT SLEEP

The deadlock happens every time.

MAKE REGRESS

Each hard-case test fails if we remove its fix.

Live demo

1

bank alone stops forever

`bank → exit 124`

2

same command + LD_PRELOAD ⟶ report

`LD_PRELOAD=./build/libdetect.so`

3

picture of the loop

`DD_DOT_OUT=cycle.dot`

4

same lock order ⟶ finishes, no report

`no_deadlock → exit 0`

RESULTS

It works. It has a cost.

PROGRAM	WITHOUT TOOL	WITH LD_PRELOAD
bank	hangs	hangs + report
sample_webapp	hangs	hangs + report
cycle3	hangs	report: 3 threads
two_pairs	hangs	2 reports, 1 per loop
no_deadlock	exit 0	exit 0, no report
lockorder	exit 0	exit 0 + order warning

20 / 20

runs found the deadlock
(bank and sample_webapp)

cost per lock + unlock

1 thread	2.3 → 72.1 ns ×31.3
4 threads, 1 lock	39.9 → 313.5 ns ×7.9

Good for finding bugs. Too slow to leave on all the time.

LIMITS & EARLY WARNING

What it can and cannot do

LIMIT

Normal mutex only

No condition variables, rwlock or semaphore.

LIMIT

Finds it after

A deadlock is found after it happens. It does not stop it.

ALSO

Warns early

Two locks taken in opposite orders → warning before a deadlock.

SUMMARY

Three things to remember

1. A deadlock is a **loop** in the wait-for graph.
2. Our tool finds it with **no code change** and shows the exact lines.
3. The fix: **lock in the same order.**

DETECTION
this tool

PREVENTION
same lock order (demo step 4) + early warning

AVOIDANCE
our Banker's simulator · docs/banker.html

DEADLOCK DETECTOR

Thank you. Questions?

03603332 Operating Systems · Group Project

Supawit Chaiyanusak
6730300591

Chayanin Iamsakul
6730300094

Pattanadech Mahimuang
6730300850

Thatthorn Jiamton
6730300779

Phutawan Supkerd
6730300884